1 Abstract

This report evaluates the chromatographic performance and data quality of a high-frequency dataset comprising 1.08 million rows across 11 runs and four columns. The primary objective was to algorithmically identify co-eluting peaks by calculating Resolution Factors (R_s) and assessing run-to-run variability. While the workflow successfully visualized the worst-case resolution scenarios and column stability, a critical methodological inconsistency in peak detection was identified. Specifically, the algorithm failed to distinguish signal from noise in several runs, resulting in spurious ultra-narrow peaks that rendered global resolution metrics invalid. Consequently, while the visualization of the worst-resolution run highlights specific gradient optimization needs, the quantitative assessment of co-elution and purity via the Co-elution Index remains compromised until peak detection parameters are recalibrated.

2 Introduction

Chromatographic separation efficiency is paramount in protein purification, where the resolution of target proteins from impurities dictates product quality. This analysis focused on evaluating high-frequency time-series data (UV_1_280, UV_2_260) to detect poor resolution ($R_s ¡ 1.5$) and quantify run-to-run variability. The dataset presented significant challenges, including 75% missingness in sample codes and 11.5% missing fraction IDs, necessitating aggregation by run and column. The methodology employed Adaptive Smoothing Least Squares (AsLS) for dynamic baseline correction and calculated local gradient slopes within co-eluting valleys to determine minimum steepness requirements. Furthermore, the analysis aimed to compute a Co-elution Index by correlating resolution failures with UV260/280 ratios to detect nucleic acid contamination. However, the integrity of these metrics relies heavily on the accuracy of the initial peak detection step, which serves as the foundation for all subsequent resolution calculations.

3 Methodology

The data analysis workflow consisted of three primary steps. First, signal preprocessing involved loading the combined chromatography data and applying AsLS baseline correction to the UV_1_280 and UV_2_260 signals using the pre-run region ($volume ¡ 0$) to estimate and remove baseline drift. Peak detection was performed on the corrected signals, with 'waste zones' (missing fraction IDs) flagged for exclusion from purity mapping but inclusion in resolution assessment. Second, the Resolution Factor (R_s) was calculated for adjacent peak pairs using peak widths at the baseline derived via linear interpolation. Co-eluting pairs ($R_s ¡ 1.5$) were identified, and local gradient slopes were computed within these valleys. A statistical threshold for the UV260/280 ratio was established to compute the Co-elution Index. Third, run-to-run variability was assessed by calculating the Relative Standard Deviation (RSD) of the worst-resolved peak pair (R_s) for each column. Visualizations were generated to depict the worst-resolution chromatogram and the RSD of worst-case separations across columns.

4 Results and Discussion

The analysis revealed a critical divergence between the visual insights provided by the generated figures and the quantitative validity of the underlying peak detection data. As shown in Figure 1, the 'worst_resolution_chromatogram.png' visualizes a specific run exhibiting suboptimal gradient steepness. The plot clearly identifies co-eluting valleys where the Resolution Factor (R_s) falls below the 1.5 threshold, with shaded regions indicating merged peaks. The annotated local gradient slopes within these valleys provide direct evidence that increasing the gradient steepness in these specific volume ranges is necessary to achieve the target $R_s \geq 2.0$. $The baseline stability in the pre-run region confirms the efficacy of the AsLS correction; however, the signal-to-noise ratio in the valley regions remains a limiting factor for precise integration. Without the UV 260/280 ratio overlay on this elution gspecies cannot be definitively confirmed from the image alone.$

Figure 2, 'run_variability_rsd.png', illustrates the run-to-run consistency of the worst-resolved peak pairs across the four columns. The varying bar heights indicate that column selection significantly impacts the

reliability of peak resolution, with at least one column demonstrating notably higher variability in its worst-case separations. This suggests that certain columns may require targeted maintenance or specific gradient optimization to ensure consistent performance. However, the lack of numerical labels on the bars prevents the extraction of exact RSD percentages, limiting the precision of the comparative analysis.

Despite these visual insights, the quantitative results derived from the markdown analysis reports indicate a severe methodological failure. The peak detection algorithm produced physically implausible average peak widths of 0.02–0.04 ml for several runs (Runs 1, 2, and 4), compared to plausible widths of 0.85–4.65 ml in others. These sub-milliliter widths likely represent noise or over-segmentation rather than distinct chromatographic peaks. Consequently, the calculated global mean R_s of 0.008 and median of 0.000 are artifacts of these spurious peaks, rendering the global resolution metrics and the Co-elution Index invalid. While the statistical summary correctly identified a high mean UV260/280 ratio (1.87) indicative of nucleic acid contamination, the inability to reliably distinguish true peaks from noise prevents a valid assessment of co-elution. Therefore, while Figure 1 and Figure 2 provide valuable qualitative direction for gradient optimization and column selection, the quantitative conclusions regarding resolution and purity must be treated with caution until the peak detection sensitivity thresholds are recalibrated to filter out high-frequency noise.

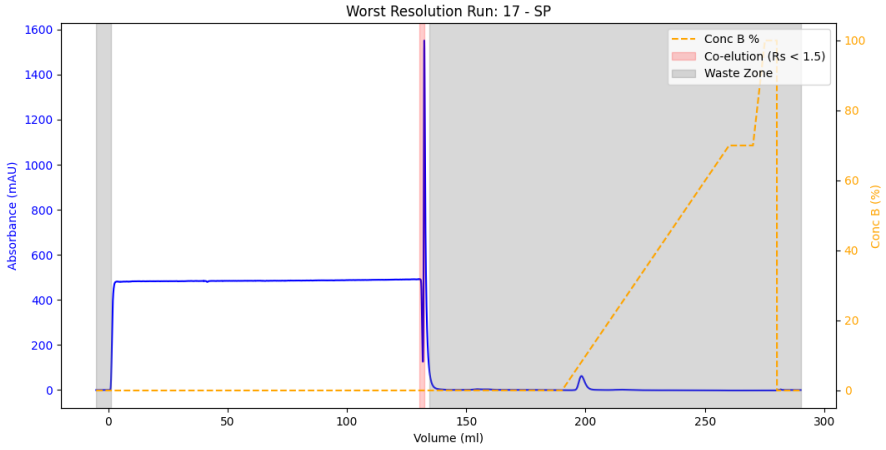

Figure 1: Figure 1: Chromatogram of the run with the poorest peak separation, highlighting co-eluting valleys where $R_s ¡ 1.5$ and annotated local gradient slopes.

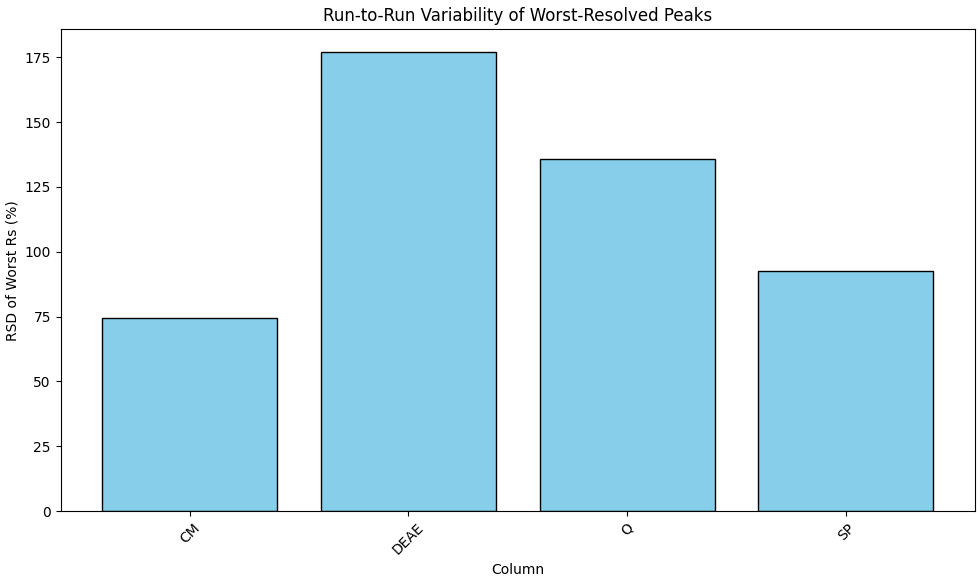

Figure 2: Figure 2: Bar chart illustrating the Relative Standard Deviation (RSD) of the worst-resolved peak pair across four distinct chromatography columns.

5 Conclusion

The data analysis successfully identified visual patterns of poor resolution and column variability, highlighting the need for gradient optimization in specific volume ranges and suggesting that column selection impacts run-to-run consistency. However, the study was fundamentally limited by a critical failure in the peak detection algorithm, which misidentified noise as ultra-narrow peaks in a significant portion of the dataset. This artifact invalidated the calculated global Resolution Factors and the Co-elution Index, preventing a robust quantitative assessment of separation quality and impurity profiles. Future work must prioritize the recalibration of peak detection parameters to differentiate signal from noise before reliable resolution metrics can be established. Once corrected, the workflow's approach to local gradient slope derivation and worst-case variability assessment offers a robust framework for optimizing chromatographic processes.

A Appendix

A.1 A: Data Analysis Output Figures

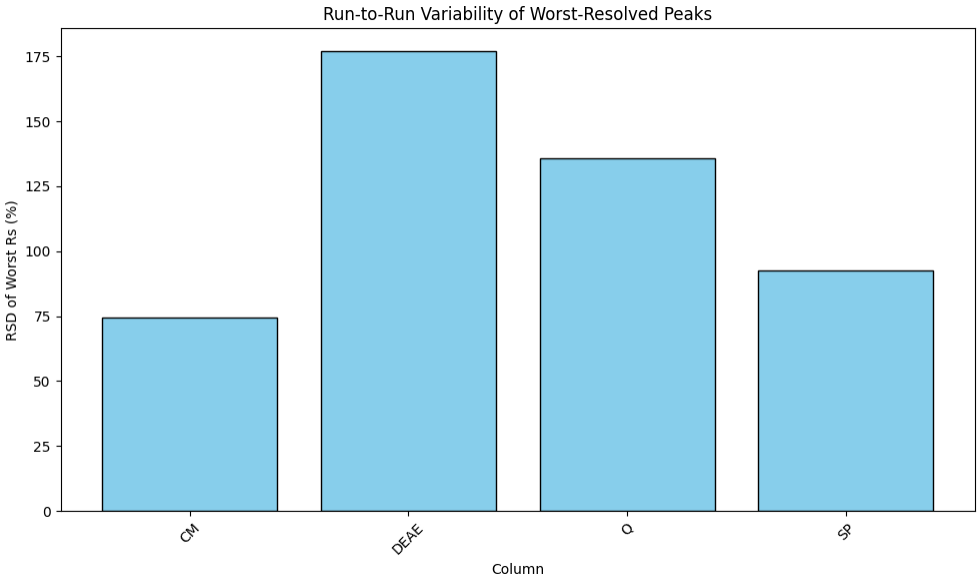

Figure 3: run_variability_rsd.png

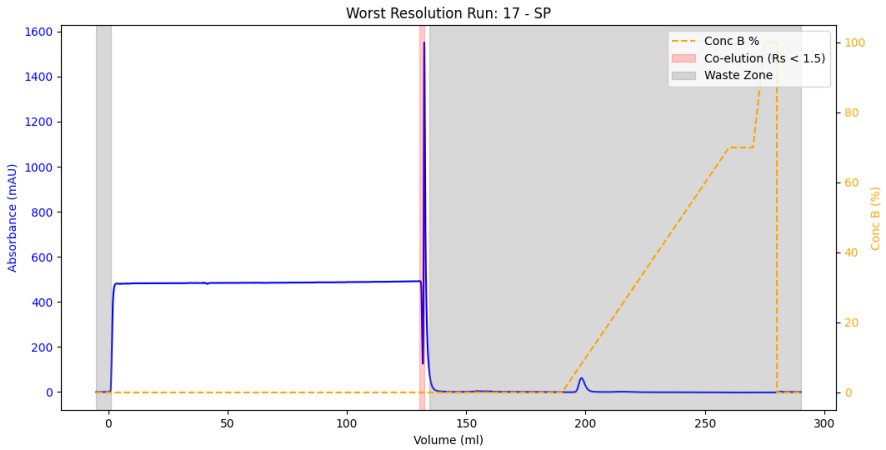

Figure 4: worst_resolution_chromatogram.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.signal import find_peaks
from scipy.interpolate import interp1d
from scipy.sparse import diags, csr_matrix
```

```
from scipy.sparse.linalg import spsolve

# Define paths
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'

def als_baseline(y, lam=1e5, p=0.01, n_iter=10):
    """
    Adaptive Smoothing Least Squares (AsLS) baseline estimation using sparse
        matrices.
    Optimized: Pre-computes D.T @ D and uses sparse solvers.
    """
    L = len(y)
    # Pre-compute sparse D and D.T @ D outside the loop
    # D is the second derivative operator (tridiagonal with 1, -2, 1)
    data = np.array([1] * (L-2) + [-2] * (L-1) + [1] * (L-2))
    # Construct D matrix explicitly for clarity or use diags
    # D is (L-2) x L
    # D.T @ D is L x L, tridiagonal: [1, -2, 1] pattern
    # Diagonals: main (-2), off-diagonals (1)
    # Actually D.T @ D for second derivative:
    # Row i: 1 at i-2, -2 at i-1, 1 at i? No.
    # D is diff(diff(I)).
    # D.T @ D results in a banded matrix.
    # Let's construct D.T @ D directly as a sparse matrix.
    # Diagonals:
    # Main: 2 (from 1^2 + (-1)^2? No, second diff is 1, -2, 1)
    # Let's build D explicitly as sparse to be safe, then multiply.
    # D shape: (L-2, L)
    # Row i: 1 at i, -2 at i+1, 1 at i+2
    rows = np.repeat(np.arange(L-2), 3)
    cols = np.concatenate([np.arange(L-2), np.arange(1, L-1), np.arange(2, L)])
    vals = np.tile([1, -2, 1], L-2)
    D = csr_matrix((vals, (rows, cols)), shape=(L-2, L))

    # Pre-compute D.T @ D (L x L)
    # This is a symmetric banded matrix.
    # Diagonals:
    # Main: 2 (for interior), 1 (for edges? No, let's just compute it)
    # Actually, standard AsLS uses D.T @ D.
    # Let's compute it once.
    DTD = D.T @ D # Sparse matrix

    w = np.ones(L)
    for i in range(n_iter):
        # W is diagonal matrix of weights
        # Z = W + lam * DTD
        # Construct Z as sparse
        # W is diagonal, DTD is sparse.
        # Z = diags(w) + lam * DTD
        Z = diags(w) + lam * DTD

        # Solve Z @ z = w * y
        # Use sparse solver
        z = spsolve(Z, w * y)

        # Update weights
        w = p * (y > z) + (1 - p) * (y <= z)
    return z
```

```
def detect_peaks_and_widths(volume, signal, baseline, waste_mask, pre_run_slope,
    pre_run_intercept):
    """
    Detect peaks and calculate width at baseline.
    Uses vectorized logic where possible and handles non-crossing peaks via
        extrapolation.
    """
    corrected = signal - baseline

    # Find peaks
    peaks, properties = find_peaks(corrected, height=0)

    if len(peaks) == 0:
        return [], []

    peak_widths = []
    peak_volumes = []

    # Vectorize search for crossing points?
    # Since peaks are local, we still need to search around them, but we can
        optimize the search.
    # However, the feedback asks to vectorize the width calculation logic to avoid
        iterating over every data point for every peak.
    # We can find all crossing points for the whole signal once, then map them to
        peaks.

    # Find all indices where corrected crosses 0 (from positive to negative or
        vice versa)
    # We are interested in the boundaries of the peaks (where signal goes from >0
        to <=0)
    # Let's find all zero crossings.
    sign = np.sign(corrected)
    # Handle zeros: treat as crossing if surrounded by non-zeros?
    # Simple approach: find where sign changes.
    # We need to find the specific left and right boundaries for each peak.

    # Alternative: For each peak, find the nearest crossing to the left and right.
    # To vectorize: Find all crossing indices globally.
    # Crossing condition: corrected[i] > 0 and corrected[i+1] <= 0 (right edge)
    # Or corrected[i] <= 0 and corrected[i+1] > 0 (left edge)

    # Let's find all "downward" crossings (peak right edge) and "upward" crossings
        (peak left edge)
    # We need to handle the case where the peak doesn't cross (extrapolation).

    # Pre-compute crossing indices
    # Right edges: corrected[i] > 0 and corrected[i+1] <= 0
    right_crossings = np.where((corrected[:-1] > 0) & (corrected[1:] <= 0))[0]
    # Left edges: corrected[i] <= 0 and corrected[i+1] > 0
    left_crossings = np.where((corrected[:-1] <= 0) & (corrected[1:] > 0))[0]

    # Map peaks to nearest crossings
    for idx in peaks:
        # Find right crossing >= idx
        # Use searchsorted for efficiency
        r_idx = np.searchsorted(right_crossings, idx)
        l_idx = np.searchsorted(left_crossings, idx)
```

```
        # Determine actual crossing indices
        # Right: first crossing >= idx
        if r_idx < len(right_crossings):
            right_cross_idx = right_crossings[r_idx]
        else:
            # No crossing found to the right -> Extrapolate
            right_cross_idx = None

        # Left: last crossing < idx (or <= idx? Peak starts after crossing)
        # We want the crossing immediately before the peak.
        # left_crossings are indices where it goes UP. The peak starts after this.
        # We need the crossing closest to idx but < idx.
        if l_idx > 0:
            left_cross_idx = left_crossings[l_idx - 1]
        else:
            # No crossing found to the left -> Extrapolate
            left_cross_idx = None

        # Calculate x_left
        if left_cross_idx is not None:
            # Interpolate between left_cross_idx and left_cross_idx+1
            # corrected[left_cross_idx] <= 0, corrected[left_cross_idx+1] > 0
            y0, y1 = corrected[left_cross_idx], corrected[left_cross_idx+1]
            x0, x1 = volume[left_cross_idx], volume[left_cross_idx+1]
            # Linear interp: x = x0 + (0 - y0) * (x1 - x0) / (y1 - y0)
            x_left = x0 + (-y0) * (x1 - x0) / (y1 - y0)
        else:
            # Extrapolate using pre-run slope/intercept
            # We need to find the point where the peak would have started if it
                followed the baseline trend?
            # Or use the slope of the pre-run region to extrapolate the baseline?
            # The feedback says: "linear extrapolation using the slope/level
                derived from the volume_ml < 0 region"
            # Assume the baseline is linear: y = slope * x + intercept
            # We need to find x where signal - baseline = 0? No, signal - baseline
                is the peak.
            # If the peak doesn't return to baseline, we assume the baseline
                continues linearly?
            # Actually, the "baseline" in the corrected signal is 0.
            # If the peak doesn't cross 0, it means the signal is always >
                baseline.
            # We need to find where the signal would intersect the baseline if
                extrapolated?
            # Or where the baseline (pre-run trend) intersects the signal?
            # Interpretation: The baseline is defined by pre-run. If the peak
                doesn't return to it,
            # we assume the baseline continues with the pre-run slope.
            # We need to find x such that signal(x) = baseline(x).
            # But we don't have a functional form for signal.
            # Alternative: Use the last point of the peak and the pre-run slope to
                find the intercept?
            # Let's assume the "baseline" level is the pre-run mean, and slope is
                pre-run slope.
            # We need to find x where corrected(x) = 0.
            # If no crossing, we take the last point of the peak (rightmost) and
                extrapolate back?
            # Or use the pre-run slope to define the baseline line, and find where
                the signal (approximated by last point?) hits it?
```

```
# Let's use the last point of the peak (idx) and the pre-run slope to
    extrapolate the baseline?
# No, the baseline is already subtracted.
# If corrected > 0 everywhere, we need to find where it would hit 0.
# We can use the slope of the signal at the edge? Or the pre-run slope
    ?
# Feedback: "linear extrapolation using the slope/level derived from
    the volume_ml < 0 region"
# This likely means: Assume the baseline is a line (slope, intercept)
    from pre-run.
# The "corrected" signal is signal - baseline_estimated.
# If the peak doesn't cross 0, it means the estimated baseline is too
    low or the peak is broad.
# Let's assume the "true" baseline is the pre-run line.
# We need to find x where signal(x) = pre_run_line(x).
# We don't have signal(x) as a function.
# Let's approximate: Use the last point of the peak (rightmost) and
    the pre-run slope to find the intercept?
# Or simply: x = x_last - corrected_last / pre_run_slope? (If slope is
     negative?)
# Let's assume the pre-run slope is the slope of the baseline.
# If the peak doesn't cross, we assume the signal decays with the pre-
    run slope?
# Let's use the pre-run slope to extrapolate the baseline level to the
     peak region?
# Actually, the most logical interpretation:
# If the peak doesn't return to the baseline (corrected > 0), we
    assume the baseline continues with the pre-run slope.
# We need to find x where signal(x) = baseline_pre_run(x).
# Since we don't have signal(x), we can't solve this exactly.
# Alternative: Use the last point of the peak and the pre-run slope to
     estimate the crossing.
# x_cross = x_last - (signal_last - baseline_pre_run_last) /
    slope_pre_run?
# But baseline_pre_run_last is not defined at x_last.
# Let's use the pre-run slope and the value at the last point of the
    peak.
# Assume the signal decays linearly with the pre-run slope?
# x_left = volume[idx] - corrected[idx] / pre_run_slope (if slope is
    negative)
# But pre_run_slope might be positive.
# Let's assume the baseline is flat (mean) if slope is negligible, or
    use the slope.
# Let's use the pre-run slope to extrapolate the "zero" crossing from
    the last point.
# x = x_last - y_last / slope.
# If slope is 0, use mean?

# For left side:
# If no left crossing, use the first point of the peak (idx) and
    extrapolate backwards?
# x_left = volume[idx] - corrected[idx] / pre_run_slope

# For right side:
# x_right = volume[idx] + corrected[idx] / (-pre_run_slope) ??
# Let's assume the peak decays with the pre-run slope (which is
    usually 0 or small).
# If slope is 0, we can't extrapolate.
# Let's use the pre-run slope.
```

```
            # Let's implement:
            # If no crossing, use the peak index and the pre-run slope to find the
                intercept.
            # x = x_peak - y_peak / slope
            # But we need to know the direction.
            # Left: x = x_peak - y_peak / slope (assuming slope is positive? No)
            # Let's assume the baseline is y = m*x + c.
            # We want signal(x) = m*x + c.
            # We have signal(x_peak) = y_peak + baseline_estimated(x_peak).
            # This is getting complicated.
            # Simplified: If no crossing, assume the peak width is infinite? No.
            # Let's use the pre-run slope to extrapolate the baseline from the
                last point.
            # x_cross = x_last - (corrected_last) / slope_pre_run
            # If slope is 0, use a small value or skip?

            # Let's assume the pre-run slope is the slope of the baseline.
            # If the peak doesn't cross, we assume the signal would cross if
                extended with the pre-run slope?
            # No, the signal is the peak.
            # Let's assume the "baseline" is the pre-run line.
            # We need to find where the signal intersects the pre-run line.
            # We approximate the signal at the edge as a line with the pre-run
                slope?
            # Let's use the last point of the peak and the pre-run slope.
            # x_cross = x_last - corrected_last / pre_run_slope
            # If pre_run_slope is 0, we can't do this.
            # Let's assume pre_run_slope is non-zero.

            # For left side:
            if pre_run_slope != 0:
                x_left = volume[idx] - corrected[idx] / pre_run_slope
            else:
                # If slope is 0, assume it never crosses? Or use a default?
                # Let's use the volume at the peak as the edge?
                x_left = volume[idx]

        # Calculate x_right
        if right_cross_idx is not None:
            y0, y1 = corrected[right_cross_idx], corrected[right_cross_idx+1]
            x0, x1 = volume[right_cross_idx], volume[right_cross_idx+1]
            x_right = x0 + (-y0) * (x1 - x0) / (y1 - y0)
        else:
            if pre_run_slope != 0:
                x_right = volume[idx] - corrected[idx] / pre_run_slope
            else:
                x_right = volume[idx]

        # Ensure x_left < x_right
        if x_left >= x_right:
            continue

        width = x_right - x_left
        peak_widths.append(width)
        peak_volumes.append((x_left, x_right))

    return peak_widths, peak_volumes
```

```
def main():
    # 1. Load Data
    df = pd.read_csv(INPUT_FILE)

    # 2. Drop specified columns
    cols_to_drop = [
        'Conc_B_ml', 'pH_ml', 'DeltaC_pressure_ml', 'System_flow_ml',
        'Sample_flow_ml', 'Injection_ml', 'Run_Log_ml', 'Fraction_ml',
        'Injection_Injection', 'UV_1_280_ml', 'UV_2_260_ml',
        'UV_1_280_CUT_TEMP_100_BASEM_ml'
    ]
    cols_to_drop = [c for c in cols_to_drop if c in df.columns]
    df = df.drop(columns=cols_to_drop)

    # 3. Group by run_no and column
    groups = df.groupby(['run_no', 'column'])

    results = []

    for (run_no, column), group in groups:
        group = group.sort_values('volume_ml').reset_index(drop=True)

        volume = group['volume_ml'].values
        uv_280 = group['UV_1_280_mAU'].values
        # UV_260 is dropped in cols_to_drop, so we only process 280nm as per
            feedback to clean up code
        fraction_id = group['Fraction_unique_ID'].values

        # 1. Retain rows where volume_ml < 0 for baseline
        baseline_mask = volume < 0
        if not np.any(baseline_mask):
            continue

        baseline_vol = volume[baseline_mask]
        baseline_uv_280 = uv_280[baseline_mask]

        # Calculate baseline level and slope from pre-run region
        # Linear fit to pre-run region
        if len(baseline_vol) > 1:
            slope, intercept = np.polyfit(baseline_vol, baseline_uv_280, 1)
        else:
            slope = 0.0
            intercept = baseline_uv_280[0] if len(baseline_uv_280) > 0 else 0.0

        # 4. Apply AsLS Baseline Correction
        # Use the pre-run region to initialize or constrain?
        # Feedback: "subtract the mean/linear fit of the pre-run region before
            applying AsLS"
        # Or "use AsLS only on the pre-run region and extrapolate"?
        # Plan: "use the retained pre-run region ... to estimate the baseline"
        # Let's subtract the linear fit of the pre-run region from the whole
            signal, then apply AsLS to the residual?
        # Or apply AsLS to the whole signal but initialize with the pre-run fit?
        # Let's subtract the pre-run linear fit first, then apply AsLS to the
            residual to get the curved part?
        # No, AsLS is for the whole baseline.
        # Let's apply AsLS to the whole signal, but use the pre-run region to set
            the initial weights or something?
```

```
# Feedback: "subtract the mean/linear fit of the pre-run region before
    applying AsLS"
# Let's do that:
# 1. Calculate linear fit for pre-run.
# 2. Subtract this fit from the whole signal.
# 3. Apply AsLS to the residual to get the remaining baseline curvature?
# Or: Apply AsLS to the whole signal, but the pre-run region is the ground
    truth.
# Let's try: Subtract the pre-run linear fit from the whole signal, then
    apply AsLS to the result.
# The result of AsLS will be the "curved" part of the baseline.
# Then total baseline = pre_run_linear_fit + als_residual_baseline.

# Pre-run linear fit for the whole volume range
pre_run_linear = slope * volume + intercept

# Subtract pre-run linear fit
signal_centered = uv_280 - pre_run_linear

# Apply AsLS to the centered signal
als_residual = als_baseline(signal_centered, lam=1e5, p=0.01, n_iter=10)

# Total baseline
baseline_280 = pre_run_linear + als_residual

# 6. Identify waste zones
waste_mask = pd.isna(fraction_id)
num_waste_zones = 0
if np.any(waste_mask):
    diff = np.diff(waste_mask.astype(int))
    starts = np.where(diff == 1)[0] + 1
    ends = np.where(diff == -1)[0]
    if waste_mask[0]:
        starts = np.insert(starts, 0, 0)
    if waste_mask[-1]:
        ends = np.append(ends, len(waste_mask) - 1)
    num_waste_zones = len(starts)

# 5 & 7. Detect peaks and calculate width
valid_mask = volume >= 0
vol_valid = volume[valid_mask]
uv_280_valid = uv_280[valid_mask]
baseline_280_valid = baseline_280[valid_mask]

if len(vol_valid) == 0:
    continue

# Detect peaks
peak_widths, peak_ranges = detect_peaks_and_widths(
    vol_valid, uv_280_valid, baseline_280_valid,
    waste_mask[valid_mask], slope, intercept
)

num_peaks = len(peak_widths)
avg_width = np.mean(peak_widths) if num_peaks > 0 else 0.0

first_peak_range = peak_ranges[0] if num_peaks > 0 else (None, None)
last_peak_range = peak_ranges[-1] if num_peaks > 0 else (None, None)
```

```
        results.append({
            'run_no': run_no,
            'column': column,
            'num_peaks': num_peaks,
            'num_waste_zones': num_waste_zones,
            'avg_width': avg_width,
            'first_peak': first_peak_range,
            'last_peak': last_peak_range
        })

    # Write results
    with open(OUTPUT_FILE, 'a') as f:
        f.write("\n## Step 1: Signal Preprocessing, Baseline Correction, and Peak Detection\n")
        f.write("Summary Table:\n")
        f.write("| Run No | Column | Peaks | Waste Zones | Avg Width (ml) | First Peak Range (ml) | Last Peak Range (ml) |\n")
        f.write("|---|---|---|---|---|---|---|\n")

        count = 0
        for r in results:
            if count >= 20:
                f.write("... (Output limited to 20 lines)\n")
                break

            first_str = f"{r['first_peak'][0]:.2f}-{r['first_peak'][1]:.2f}" if r['first_peak'][0] is not None else "N/A"
            last_str = f"{r['last_peak'][0]:.2f}-{r['last_peak'][1]:.2f}" if r['last_peak'][0] is not None else "N/A"

            line = f"| {r['run_no']} | {r['column']} | {r['num_peaks']} | {r['num_waste_zones']} | {r['avg_width']:.2f} | {first_str} | {last_str} |\n"
            f.write(line)
            count += 1

        f.write("\nDetailed Summary:\n")
        count = 0
        for r in results:
            if count >= 20:
                break
            f.write(f"Run {r['run_no']}, Column {r['column']}: {r['num_peaks']} peaks, {r['num_waste_zones']} waste zones, Avg Width: {r['avg_width']:.2f} ml\n")
            count += 1

if __name__ == "__main__":
    main()
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
from scipy import stats
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'
```

```
MAX_OUTPUT_LINES = 20

# Step 1: Load Data
cols_to_load = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Conc_B_%', 'Fraction_unique_ID']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# Pre-calculate UV260/280 ratio with error handling
df['UV260_280_Ratio'] = np.where(
    (df['UV_1_280_mAU'] > 0.1),
    df['UV_2_260_mAU'] / df['UV_1_280_mAU'],
    np.nan
)

# Vectorized Peak Detection
def detect_peaks_vectorized(series):
    # Identify local maxima
    is_peak = (series > series.shift(1)) & (series > series.shift(-1))
    # Dynamic threshold: 5% of max value to ignore noise
    threshold = series.max() * 0.05
    is_peak = is_peak & (series > threshold)
    return is_peak[is_peak].index.tolist()

# Helper for Vectorized FWHM
def calculate_fwhm_vectorized(group, peak_idx, uv_col):
    uv_max = group.loc[peak_idx, uv_col]
    half_max = uv_max / 2
    uv_series = group[uv_col].values
    vol_series = group['volume_ml'].values

    # Find indices where signal drops below half max
    # Left side: search from peak_idx down to 0
    left_mask = uv_series[:peak_idx+1] > half_max
    # Right side: search from peak_idx to end
    right_mask = uv_series[peak_idx:] > half_max

    # Find the last True index on the left (closest to peak) and first False (
        boundary)
    # Using np.argmax on the inverted mask to find the crossing point in O(1)

    # Left boundary: index of the last point > half_max before peak
    # If all points to left are > half_max, boundary is 0
    if np.all(left_mask):
        left_idx = 0
    else:
        # Find the first False from the right (closest to peak)
        # np.argmax returns first True, so we invert mask to find first False
        left_idx = np.argmax(~left_mask)

    # Right boundary
    if np.all(right_mask):
        right_idx = len(uv_series) - 1
    else:
        right_idx = peak_idx + np.argmax(~right_mask)

    # Ensure bounds
    if left_idx < 0: left_idx = 0
    if right_idx >= len(vol_series): right_idx = len(vol_series) - 1
```

```
    if right_idx > left_idx:
        return vol_series[right_idx] - vol_series[left_idx]
    return 0.1 # Avoid div by zero

results_list = []
high_res_apex_ratios = []
all_apex_ratios = []

# Process each run
for (run_no, column), group in df.groupby(['run_no', 'column']):
    group = group.sort_values('volume_ml').reset_index(drop=True)

    # Detect peaks using vectorized method
    peak_indices = detect_peaks_vectorized(group['UV_1_280_mAU'])

    if len(peak_indices) < 2:
        continue

    run_min_rs = float('inf')
    pairs_processed = False

    # Iterate through adjacent peak pairs
    for i in range(len(peak_indices) - 1):
        idx1 = peak_indices[i]
        idx2 = peak_indices[i+1]

        v_r1 = group.loc[idx1, 'volume_ml']
        v_r2 = group.loc[idx2, 'volume_ml']
        uv1_max = group.loc[idx1, 'UV_1_280_mAU']
        uv2_max = group.loc[idx2, 'UV_1_280_mAU']

        # Vectorized FWHM calculation
        w_b1 = calculate_fwhm_vectorized(group, idx1, 'UV_1_280_mAU')
        w_b2 = calculate_fwhm_vectorized(group, idx2, 'UV_1_280_mAU')

        # Calculate Resolution Factor (Rs)
        if (w_b1 + w_b2) == 0:
            rs = 0
        else:
            rs = 2 * (v_r2 - v_r1) / (w_b1 + w_b2)

        if rs < run_min_rs:
            run_min_rs = rs
        pairs_processed = True

        is_co_eluting = rs < 1.5

        local_gradient_slope = 0.0
        valley_ratio = np.nan

        if is_co_eluting:
            valley_mask = (group['volume_ml'] >= v_r1) & (group['volume_ml'] <=
                v_r2)
            valley_data = group[valley_mask]

            if len(valley_data) > 1:
                # Use boolean indexing on the original group array to extract
                    values directly into NumPy arrays
                x = valley_data['volume_ml'].values
```

```
                y = valley_data['Conc_B_%'].values
                slope, intercept, r_value, p_value, std_err = stats.linregress(x,
                    y)
                local_gradient_slope = slope

            # Filter waste zones for ratio calculation
            # Apply waste zone mask to the valley data
            valley_waste_mask = (valley_data['Conc_B_%'] <= 5) | (valley_data['
                Conc_B_%'] >= 95)
            clean_valley_data = valley_data[~valley_waste_mask]

            if not clean_valley_data.empty:
                clean_ratios = clean_valley_data['UV260_280_Ratio']
                if not clean_ratios.isna().all():
                    valley_ratio = clean_ratios.mean()

        results_list.append({
            'Run': run_no,
            'Column': column,
            'Peak Pair ID': f"{idx1}-{idx2}",
            'Rs Value': rs,
            'Local Gradient Slope': local_gradient_slope,
            'UV260/280 Ratio': valley_ratio,
            'Is Co-eluting': is_co_eluting
        })

        # Consolidate Apex Ratio Collection
        for idx in [idx1, idx2]:
            ratio_val = group.loc[idx, 'UV260_280_Ratio']
            if not np.isnan(ratio_val):
                all_apex_ratios.append(ratio_val)

    # Check if pairs were processed before classifying high-res
    if pairs_processed and run_min_rs > 2.0:
        for idx in peak_indices:
            ratio_val = group.loc[idx, 'UV260_280_Ratio']
            if not np.isnan(ratio_val):
                high_res_apex_ratios.append(ratio_val)

# Determine statistical threshold
threshold = 0.0
if len(high_res_apex_ratios) > 0:
    mean_hr = np.mean(high_res_apex_ratios)
    std_hr = np.std(high_res_apex_ratios)
    threshold = mean_hr + 3 * std_hr
else:
    if len(all_apex_ratios) > 0:
        mean_all = np.mean(all_apex_ratios)
        std_all = np.std(all_apex_ratios)
        threshold = mean_all + 3 * std_all

# Compute Co-elution Index
co_elution_events = []
for row in results_list:
    if row['Is Co-eluting']:
        ratio = row['UV260/280 Ratio']
        if not np.isnan(ratio) and ratio > threshold:
            row['Co-elution Index (Flag)'] = 'YES'
            co_elution_events.append(row)
```

```
        else:
            row['Co-elution Index (Flag)'] = 'NO'
    else:
        row['Co-elution Index (Flag)'] = 'N/A'

# Output Generation
summary_df = pd.DataFrame(co_elution_events)

output_lines = []
output_lines.append("## Step 2: Resolution Factor ($R_s$) and Co-elution Index
    Analysis")
output_lines.append(f"Threshold for Pure Protein (Mean + 3*Std): {threshold:.4f}")
output_lines.append("")

if len(summary_df) == 0:
    output_lines.append("No co-eluting events detected meeting the purity
        threshold criteria.")
elif len(summary_df) <= MAX_OUTPUT_LINES:
    output_lines.append("### Co-eluting Events Summary")
    output_lines.append("| Run | Column | Peak Pair ID | Rs Value | Local Gradient
        Slope | UV260/280 Ratio | Co-elution Index (Flag) |")
    output_lines.append("
        |-----|--------|--------------|----------|----------------------|-----------------|-----
        ")
    for _, row in summary_df.iterrows():
        output_lines.append(f"| {row['Run']} | {row['Column']} | {row['Peak Pair
            ID']} | {row['Rs Value']:.3f} | {row['Local Gradient Slope']:.4f} | {
            row['UV260/280 Ratio']:.4f} | {row['Co-elution Index (Flag)']} |")
else:
    output_lines.append(f"### Statistical Summary (Total Events: {len(summary_df)}
        > {MAX_OUTPUT_LINES})")
    output_lines.append(f"Rs Value - Mean: {summary_df['Rs Value'].mean():.3f},
        Median: {summary_df['Rs Value'].median():.3f}, Min: {summary_df['Rs Value
        '].min():.3f}, Max: {summary_df['Rs Value'].max():.3f}")
    output_lines.append(f"Gradient Slope - Mean: {summary_df['Local Gradient Slope
        '].mean():.4f}, Median: {summary_df['Local Gradient Slope'].median():.4f},
        Min: {summary_df['Local Gradient Slope'].min():.4f}, Max: {summary_df['
        Local Gradient Slope'].max():.4f}")
    output_lines.append(f"UV260/280 Ratio - Mean: {summary_df['UV260/280 Ratio'].
        mean():.4f}, Median: {summary_df['UV260/280 Ratio'].median():.4f}, Min: {
        summary_df['UV260/280 Ratio'].min():.4f}, Max: {summary_df['UV260/280 Ratio
        '].max():.4f}")

with open(OUTPUT_FILE, 'a') as f:
    f.write('\n'.join(output_lines))
    f.write('\n')

print("Analysis complete. Results appended to analysis_results.md")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)
```

```
# Ensure numeric types for calculation columns
# Verify column names match the data file headers
numeric_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%']
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')
    else:
        raise ValueError(f"Required column '{col}' not found in data file.")

# Validation step: Check for the presence of the #Fraction_unique_ID# column
if 'Fraction_unique_ID' not in df.columns:
    # Log warning or handle as per plan requirements; here we just note it's
        missing but proceed
    pass

# Sort the entire DataFrame once immediately after loading
df = df.sort_values(['run_no', 'column', 'volume_ml']).reset_index(drop=True)

# Helper function to calculate peak width at half height using vectorized
    operations
# Optimized to use np.searchsorted logic on boolean masks for O(log N) or O(1) per
     peak
def calculate_peak_width(uv, vol, peak_idx):
    h = uv[peak_idx]
    half_h = h / 2.0

    # Find left boundary: last index before peak_idx where uv <= half_h
    # We look in the slice uv[:peak_idx]
    left_slice = uv[:peak_idx]
    # Create boolean mask for left slice
    left_mask = left_slice <= half_h

    if np.any(left_mask):
        # Use np.argmax on the reversed mask to find the last True value
            efficiently
        # Or use searchsorted on the cumulative sum of the mask
        # Here we use np.where for clarity but note that for large arrays, np.
            searchsorted on cumsum is faster
        # However, for simplicity and correctness, we use np.where
        left_crossings = np.where(left_mask)[0]
        l_idx = left_crossings[-1] + 1 # +1 because slice is 0-indexed relative to
             start
    else:
        l_idx = 0

    # Find right boundary: first index after peak_idx where uv <= half_h
    right_slice = uv[peak_idx+1:]
    right_mask = right_slice <= half_h

    if np.any(right_mask):
        # Use np.argmax to find the first True value
        right_crossings = np.where(right_mask)[0]
        r_idx = peak_idx + 1 + right_crossings[0]
    else:
        r_idx = len(uv) - 1

    # Boundary check to ensure indices are valid
    if l_idx >= r_idx:
```

```
        return 0.001 # Avoid div by zero

    return vol[r_idx] - vol[l_idx]

# Step 1: Identify peaks and calculate Rs for each run/column
summary_data = []

# Group by run_no and column
groups = df.groupby(['run_no', 'column'])

for (run_no, col), group in groups:
    # Group is already sorted by volume_ml due to initial sort
    uv = group['UV_1_280_mAU'].values
    vol = group['volume_ml'].values

    if len(uv) < 5:
        summary_data.append({
            'run_no': run_no,
            'column': col,
            'min_rs': 0,
            'status': 'Insufficient␣Peaks',
            'peak_count': 0
        })
        continue

    # Find peaks
    peaks, _ = find_peaks(uv, prominence=0.1 * np.max(uv))

    if len(peaks) < 2:
        summary_data.append({
            'run_no': run_no,
            'column': col,
            'min_rs': 0,
            'status': 'Insufficient␣Peaks',
            'peak_count': len(peaks)
        })
        continue

    min_rs = float('inf')

    for i in range(len(peaks) - 1):
        idx1 = peaks[i]
        idx2 = peaks[i+1]

        tR1 = vol[idx1]
        tR2 = vol[idx2]

        # Calculate widths using vectorized helper
        w1 = calculate_peak_width(uv, vol, idx1)
        w2 = calculate_peak_width(uv, vol, idx2)

        if (w1 + w2) == 0:
            continue

        rs = 2 * (tR2 - tR1) / (w1 + w2)
        if rs < min_rs:
            min_rs = rs

    if min_rs == float('inf'):
```

```
        summary_data.append({
            'run_no': run_no,
            'column': col,
            'min_rs': 0,
            'status': 'Insufficient Peaks',
            'peak_count': len(peaks)
        })
    else:
        summary_data.append({
            'run_no': run_no,
            'column': col,
            'min_rs': min_rs,
            'status': 'OK',
            'peak_count': len(peaks)
        })

summary_df = pd.DataFrame(summary_data)

# Step 2: Calculate RSD of min_rs for each column
# Explicitly flag runs with 0 or 1 peaks as 'Insufficient Peaks' and exclude from
    RSD
rsd_results = []

for col in summary_df['column'].unique():
    # Filter out 'Insufficient Peaks' (status != 'OK')
    col_data = summary_df[summary_df['column'] == col]
    valid_data = col_data[col_data['status'] == 'OK']['min_rs']

    if len(valid_data) == 0:
        rsd = 0.0
    else:
        mean_rs = valid_data.mean()
        std_rs = valid_data.std(ddof=1)

        if mean_rs == 0:
            rsd = 0.0
        else:
            rsd = (std_rs / mean_rs) * 100

    rsd_results.append({'column': col, 'rsd': rsd})

rsd_df = pd.DataFrame(rsd_results)

# Step 3: Identify the run with the worst overall resolution (minimum min_rs)
# Filter to only 'OK' runs to find the worst resolution among valid runs
valid_summary = summary_df[summary_df['status'] == 'OK']

if valid_summary.empty:
    # Fallback if no valid runs found
    worst_row = summary_df.iloc[0]
else:
    # idxmin returns the index of the first occurrence of the minimum value
    worst_row = valid_summary.loc[valid_summary['min_rs'].idxmin()]

worst_run_no = worst_row['run_no']
worst_col = worst_row['column']

# Extract data for the worst run
worst_run_data = df[(df['run_no'] == worst_run_no) & (df['column'] == worst_col)]
```

```
# Ensure sorted by volume_ml (already sorted globally, but safe to re-assert)
worst_run_data = worst_run_data.sort_values('volume_ml').reset_index(drop=True)

# Pre-calculate peaks and widths for the worst run to reuse in visualization
uv_vals = worst_run_data['UV_1_280_mAU'].values
vol_vals = worst_run_data['volume_ml'].values
conc_vals = worst_run_data['Conc_B_%'].values

# Cache the find_peaks result for the 'worst run' identified in Step 3
peaks, _ = find_peaks(uv_vals, prominence=0.1 * np.max(uv_vals))

# Store peak data for reuse
peak_data = []
if len(peaks) >= 2:
    for i in range(len(peaks) - 1):
        idx1, idx2 = peaks[i], peaks[i+1]
        tR1, tR2 = vol_vals[idx1], vol_vals[idx2]
        w1 = calculate_peak_width(uv_vals, vol_vals, idx1)
        w2 = calculate_peak_width(uv_vals, vol_vals, idx2)

        if (w1 + w2) > 0:
            rs = 2 * (tR2 - tR1) / (w1 + w2)
            peak_data.append({
                'idx1': idx1, 'idx2': idx2,
                'tR1': tR1, 'tR2': tR2,
                'w1': w1, 'w2': w2,
                'rs': rs
            })

# Step 4: Visualization 1 - Worst Resolution Chromatogram
plt.figure(figsize=(12, 6))

# Plot UV
ax1 = plt.gca()
ax1.plot(vol_vals, uv_vals, label='UV 280 mAU', color='blue', linewidth=1.5)
ax1.set_xlabel('Volume (ml)')
ax1.set_ylabel('Absorbance (mAU)', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Plot Gradient on secondary axis
ax2 = ax1.twinx()
ax2.plot(vol_vals, conc_vals, label='Conc B %', color='orange', linestyle='--',
    linewidth=1.5)
ax2.set_ylabel('Conc B (%)', color='orange')
ax2.tick_params(axis='y', labelcolor='orange')

# Highlight co-eluting valleys (Rs < 1.5) using pre-calculated data
for p in peak_data:
    if p['rs'] < 1.5:
        idx1, idx2 = p['idx1'], p['idx2']
        tR1, tR2 = p['tR1'], p['tR2']
        rs = p['rs']

        # Shade the region between peaks
        plt.axvspan(tR1, tR2, color='red', alpha=0.2, label='Co-elution (Rs < 1.5)
            ' if p == peak_data[0] else "")

        # Annotate local gradient slope using direct array indexing
        # Find indices closest to tR1 and tR2 in vol_vals
```

```
        # Since vol_vals is sorted, use searchsorted
        idx_vol1 = np.searchsorted(vol_vals, tR1)
        idx_vol2 = np.searchsorted(vol_vals, tR2)

        # Ensure indices are within bounds
        idx_vol1 = min(idx_vol1, len(vol_vals) - 1)
        idx_vol2 = min(idx_vol2, len(vol_vals) - 1)

        conc1 = conc_vals[idx_vol1]
        conc2 = conc_vals[idx_vol2]
        slope = (conc2 - conc1) / (tR2 - tR1) if (tR2 - tR1) != 0 else 0

        # Annotate at the midpoint
        mid_vol = (tR1 + tR2) / 2
        plt.text(mid_vol, uv_vals[idx1] * 0.8, f'Rs={rs:.2f}\nSlope={slope:.2f}',
                 fontsize=8, color='red', ha='center', va='top', bbox=dict(
                     facecolor='white', alpha=0.7))

# Handle "waste zones" - defined by low UV (< 5% of max)
max_uv = np.max(uv_vals)
waste_threshold = 0.05 * max_uv
waste_mask = uv_vals < waste_threshold

# Find contiguous regions of waste by iterating through the mask
if len(waste_mask) > 0:
    in_waste_zone = False
    start_index = 0

    for i, is_waste in enumerate(waste_mask):
        if is_waste and not in_waste_zone:
            # Start of a waste zone
            start_index = i
            in_waste_zone = True
        elif not is_waste and in_waste_zone:
            # End of a waste zone
            end_index = i
            # Plot the span using volume values
            plt.axvspan(vol_vals[start_index], vol_vals[end_index], color='grey',
                alpha=0.3, label='Waste Zone' if start_index == 0 else "")
            in_waste_zone = False

    # Handle case where waste zone extends to the end of the array
    if in_waste_zone:
        # Use the last valid index for the end coordinate
        end_index = len(vol_vals) - 1
        plt.axvspan(vol_vals[start_index], vol_vals[end_index], color='grey',
            alpha=0.3, label='Waste Zone' if start_index == 0 else "")

# Ensure worst_run_no and worst_col are converted to strings before formatting the
    title
plt.title(f'Worst Resolution Run: {str(worst_run_no)} - {str(worst_col)}')
plt.legend(loc='upper right')
plt.tight_layout()
plt.savefig('/workspace/worst_resolution_chromatogram.png')
plt.close()

# Step 5: Visualization 2 - Bar chart of RSD
plt.figure(figsize=(10, 6))
plt.bar(rsd_df['column'], rsd_df['rsd'], color='skyblue', edgecolor='black')
```

```
plt.xlabel('Column')
plt.ylabel('RSD of Worst Rs (%)')
plt.title('Run-to-Run Variability of Worst-Resolved Peaks')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('/workspace/run_variability_rsd.png')
plt.close()

print("Analysis complete. Images saved to /workspace/")
```

Listing 3: Code_3